DOCUMENTO TÉCNICO · ARQUITECTURA

Plataforma de Chat Multiagente y Multicanal

Arquitectura de servicios, capas, flujo de conversación y estado real de implementación — WhatsApp, Messenger, Instagram y Webchat, con derivación a equipos, atención humana y paneles de administración.

Versión — revisión completa, reemplaza `arquitectura-y-frontends-nuevos.pdf` (sep. 2026)

Fecha — octubre 2026

Alcance — Core, Channel Gateway, Conversation Engine, Webchat, Admin Panel, Operators Desk Web, App Android

Documento complementario — "Qué falta para producción" (pendientes de hardening, observabilidad y despliegue)

Índice

1. Qué es la plataforma y cómo leer este documento
2. Vista de capas — arquitectura completa, de los canales a la base de datos
3. Catálogo de servicios y su estado real
4. Flujo de una conversación, de punta a punta — del primer mensaje del contacto a la conversación resuelta
5. Autenticación, autorización y multi-tenancy
6. Modelo de datos por servicio — resumen de entidades
7. Convenciones multicanal — selección nativa y formato neutro
8. Estado de implementación por fase
9. Glosario

01

Qué es la plataforma y cómo leer este documento

Es una plataforma **multi-tenant** de atención por chat: cada organización cliente administra sus propios equipos, horarios, árbol de menú del bot y canales (WhatsApp, Messenger, Instagram y un widget de chat embebible en su propio sitio), y sus operadores atienden esas conversaciones una vez que el bot las deriva a un equipo. Un contacto nunca habla con "la plataforma" en abstracto: siempre está dentro del árbol de menú de una organización concreta, y todo dato queda aislado por `organization_id` en cada tabla de negocio.

El principio que explica casi todas las decisiones de diseño de abajo es simple: **Core decide, todo lo demás ejecuta.** Toda la lógica de negocio (qué responde el bot, a qué equipo deriva, qué operador puede ver qué, si está en horario de atención) vive en un único servicio Django (`services/core`). Los demás servicios son ejecutores de I/O — reciben un webhook, abren un WebSocket, pegan a la Graph API de Meta — y le preguntan a Core "¿qué hago con esto?" en vez de decidirlo ellos.

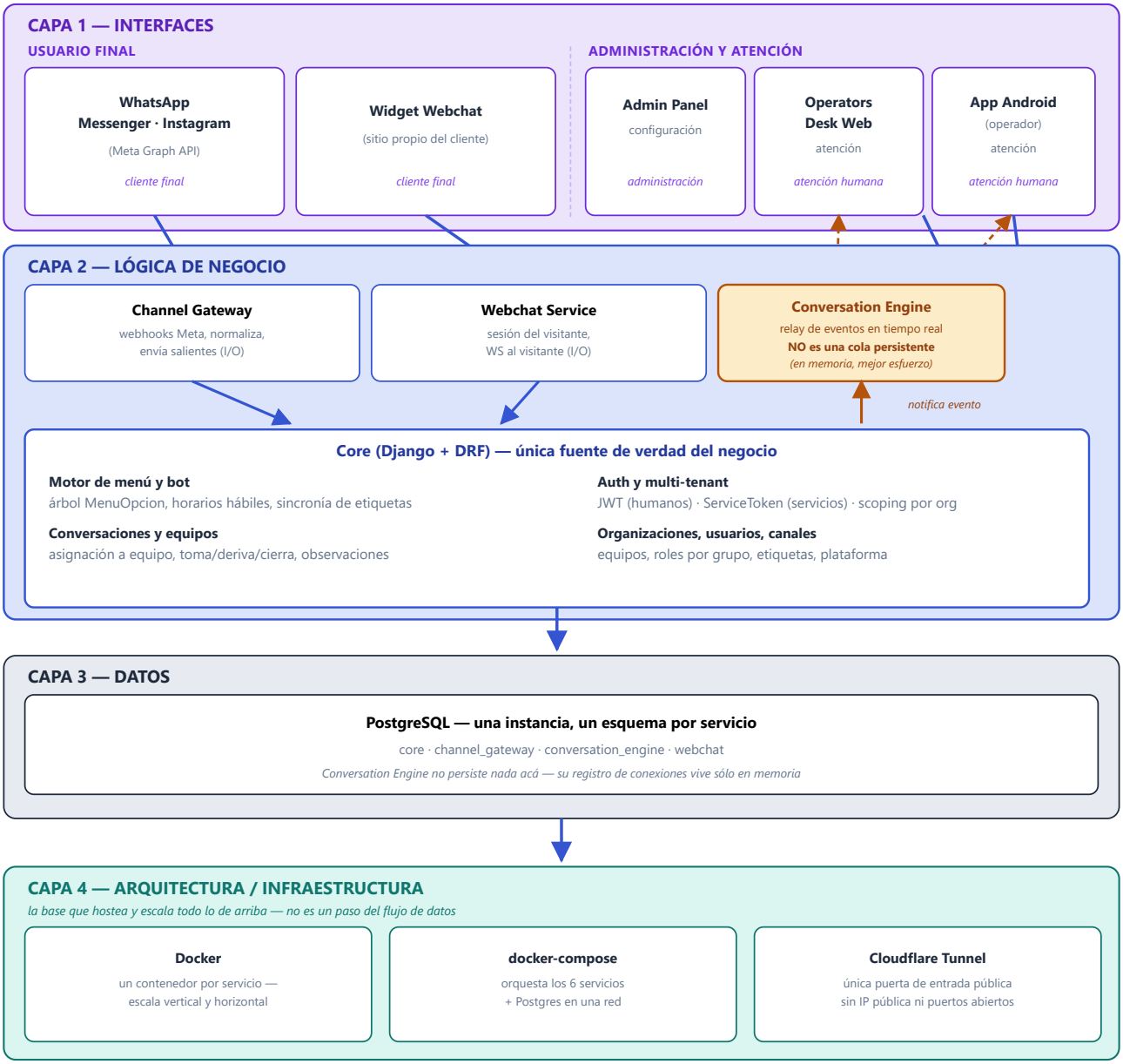
Este documento describe la arquitectura *tal como está construida hoy*, no un plan a futuro. Donde algo sigue siendo un stub o está parcialmente implementado, se dice explícitamente (sección 3). Los pendientes de hardening, observabilidad y despliegue a producción se separaron a propósito a un documento aparte para no mezclar "cómo está hecho" con "qué falta para confiar en esto con usuarios reales".

Es un **MVP construido para escalar, no un prototipo descartable**: cada servicio corre en su propio contenedor Docker (un `Dockerfile` por servicio, orquestados con `docker-compose`) precisamente para poder escalar cada pieza por separado el día que el volumen real lo pida — **verticalmente** (más CPU/memoria a un contenedor puntual bajo carga, ej. Core) y **horizontalmente** (más réplicas de un mismo servicio detrás de un balanceador, ej. varias instancias de Channel Gateway en un pico de mensajes de WhatsApp) sin tener que escalar el sistema completo de una sola vez. Por eso microservicios con fronteras claras y un esquema de base propio por servicio desde el día uno, en vez de un monolito único.

02

Vista de capas

Cuatro capas clásicas, de arriba hacia abajo: **interfaces** (por dónde entra y sale todo — tanto el cliente final como quien administra o atiende), **lógica de negocio** (quién decide y quién ejecuta I/O), **datos** (dónde se guarda), y **arquitectura/infraestructura** (dónde corre todo lo anterior y por qué está dockerizado). Las flechas sólidas son llamadas directas (REST/HTTP); las punteadas son notificaciones de mejor esfuerzo (WebSocket), nunca el camino principal de ningún dato.



Camino de entrada (mensaje de un contacto):
Usuario final (capa 1) → Channel Gateway / Webchat Service normalizan (capa 2) → Core decide qué responder y a qué equipo deriva (capa 2) → Core guarda en Postgres (capa 3).

Cómo se entera un operador (en el medio, entre el usuario y el operador):
Core le avisa a Conversation Engine (mejor esfuerzo) → Conversation Engine reenvía por WebSocket a quien ya esté conectado en Operators Desk Web / App Android (capa 1). **No hay cola**: si nadie está conectado en ese momento, no se guarda nada para después — el operador recién se entera cuando su pantalla haga el próximo polling de respaldo (~10-12s). Por eso en la sección 3 figura como "parcial": cumple su función hoy, pero sin la durabilidad de una cola de verdad.

Cómo actúa un operador (acciones, no notificaciones):
Operators Desk Web / App Android (capa 1) → REST directo a Core (capa 2, `tomar/responder/derivar/cerrar/etiquetar`) — nunca pasa por Conversation Engine, que sólo notifica, no ejecuta nada.

Camino de salida (la respuesta llega al contacto):
Core decide el canal según `conversacion.canal` (capa 2) → despacha al cliente saliente correcto (`channel_gateway_client.py` o `webchat_client.py`) → Meta Graph API o el WebSocket del visitante entregan al usuario final (capa 1).

Autenticación entre servicios (nunca el JWT de una persona):
`Authorization: Service-Token <token>` — un secreto por servicio, cifrado en reposo (Fernet).

Figura 1 — Arquitectura en 4 capas. Conversation Engine vive en la capa de lógica de negocio, a mitad de camino entre el mensaje que entra del usuario final y la bandeja del operador — pero como relay de notificaciones en tiempo real, no como una cola con persistencia.

SOBRE CONVERSATION ENGINE — LA PREGUNTA MÁS COMÚN DE ESTE DOCUMENTO

Conceptualmente sí está "en el medio" entre el usuario final y el operador, como una cola de mensajería clásica — pero **no está implementado como una cola**: no tiene persistencia, no reintenta, no tiene backlog. Es un registro de conexiones WebSocket en memoria de un solo proceso (`app/connections.py`) al que Core le avisa "pasó algo" por `POST /internal/eventos/` y que reenvía ese aviso a quien ya esté escuchando — si nadie escucha en ese instante, el aviso se pierde sin dejar rastro, y el operador se entera igual más tarde por el polling de respaldo que corre en Operators Desk Web/App Android. Es la pieza más frágil de la arquitectura hoy (ver estado "Parcial" en la sección 3 y el detalle en "Qué falta para producción") — una cola de verdad (con Postgres LISTEN/NOTIFY, no Redis/Kafka por la restricción de stack) es el camino natural si el volumen lo pide.

LECTURA CLAVE DE LA FIGURA

La capa de lógica de negocio es la única que decide — las capas de interfaces no tienen fuente de verdad propia (ni siquiera Channel Gateway o Webchat Service, que sólo ejecutan I/O). Si algún día una interfaz necesitara decidir algo de negocio por su cuenta, es señal de que falta un endpoint en Core — regla aplicada sin excepciones durante todo el desarrollo.

Exposición pública real

Cada contenedor es internamente accesible sólo dentro de la red de `docker-compose` — lo único expuesto a internet pasa por un Cloudflare Tunnel (sin IP pública ni puertos abiertos en el servidor), un hostname propio por servicio que necesita ser alcanzable desde afuera:

Dominio	Apunta a	Para qué
`api.udcchat.com.ar`	Core	API que consumen el Admin Panel, Operators Desk Web y la app Android
`admin.udcchat.com.ar`	Admin Panel	Panel de configuración (superusuario/administrador)
`operadores.udcchat.com.ar`	Operators Desk Web	Atención de conversaciones desde el navegador
`realtime.udcchat.com.ar`	Conversation Engine	WebSocket de entrega en tiempo real a operadores
`webhooks.udcchat.com.ar`	Channel Gateway	Webhooks entrantes de WhatsApp/Messenger/Instagram
`webchat.udcchat.com.ar`	Webchat Service	Widget embebible + su WebSocket al visitante

03

Catálogo de servicios y su estado real

Tamaño de código (líneas de Python/Java, sin tests) y cobertura de tests como señal objetiva de madurez — no todo lo que figura en el diagrama de capas está igual de construido.

Implementado y en uso Parcial / primera versión

Servicio	Stack · puerto	Qué hace hoy	Código	Tests	Estado
Core	Django+DRF :8001	Organizaciones, usuarios/grupos, equipos, horarios, árbol de menú/bot, conversaciones, etiquetas, canales, tokens internos, sección Plataforma. Fuente de verdad de todo.	~12 100 líneas	12 módulos de test	Implementado
Channel Gateway	FastAPI :8002	Webhooks de WhatsApp/Messenger/Instagram (verificación de firma, normalización), envío de salientes vía Graph API, traducción a selección nativa (listas/botones).	~2 000 líneas	6 módulos	Implementado
Conversation Engine	FastAPI + WS :8003	Entrega en tiempo real a operadores conectados (`WS /ws/operador/{token}`). Sin tablas propias de negocio — registro de conexiones en memoria de un solo proceso.	~660 líneas	4 módulos	Parcial
Webchat Service	FastAPI + WS :8006	Primera versión: sesión del visitante, mensaje entrante/saliente con el mismo contrato que Channel Gateway, WebSocket al visitante, `widget.js` embebible. Sin Postgres propio — todo en memoria de proceso. Opt-in (`--profile webchat`).	~450 líneas	6 módulos (37 tests)	Implementado
Admin Panel	React+TS :3000	Login, usuarios/equipos/horarios/menú/mensajes/etiquetas/canales de la propia organización, y sección "Plataforma" completa para el superusuario real.	19 páginas	—	Implementado
Operators Desk Web (React)	React+TS :3001	Bandeja/hilo de conversación, tomar/derivar/etiquetar/cerrar con observación, notas internas, perfil — mismo contrato de API que la app Android. Habla directo contra Core (`api.udcchat.com.ar`). Publicada en `operadores.udcchat.com.ar` .	~1 700 líneas	0 módulos	Implementado
App Android (operador)	Java, MVVM	Login, bandeja, hilo de conversación, tomar/derivar/etiquetar/cerrar con observación, notas internas, equipos en el perfil, polling + WebSocket de Conversation Engine.	20 clases Java	—	Implementado

LECTURA CLAVE DE LA TABLA

"Operators Desk" existió como un servicio FastAPI aparte (puerto 8004), pensado como BFF para Operators Desk Web — nunca tuvo un consumidor real (la app React termina hablándole directo a Core) y se dio de baja del repo y del `docker-compose.yml` (2026-10). Operators Desk Web es la única pieza real de esta fase hoy. Un pendiente genuino, sí: no está en `docker-compose.yml` (vive en `services/operators-desk/web/` pero se construye/despliega aparte) — un `docker compose up` de cero hoy no la levanta.

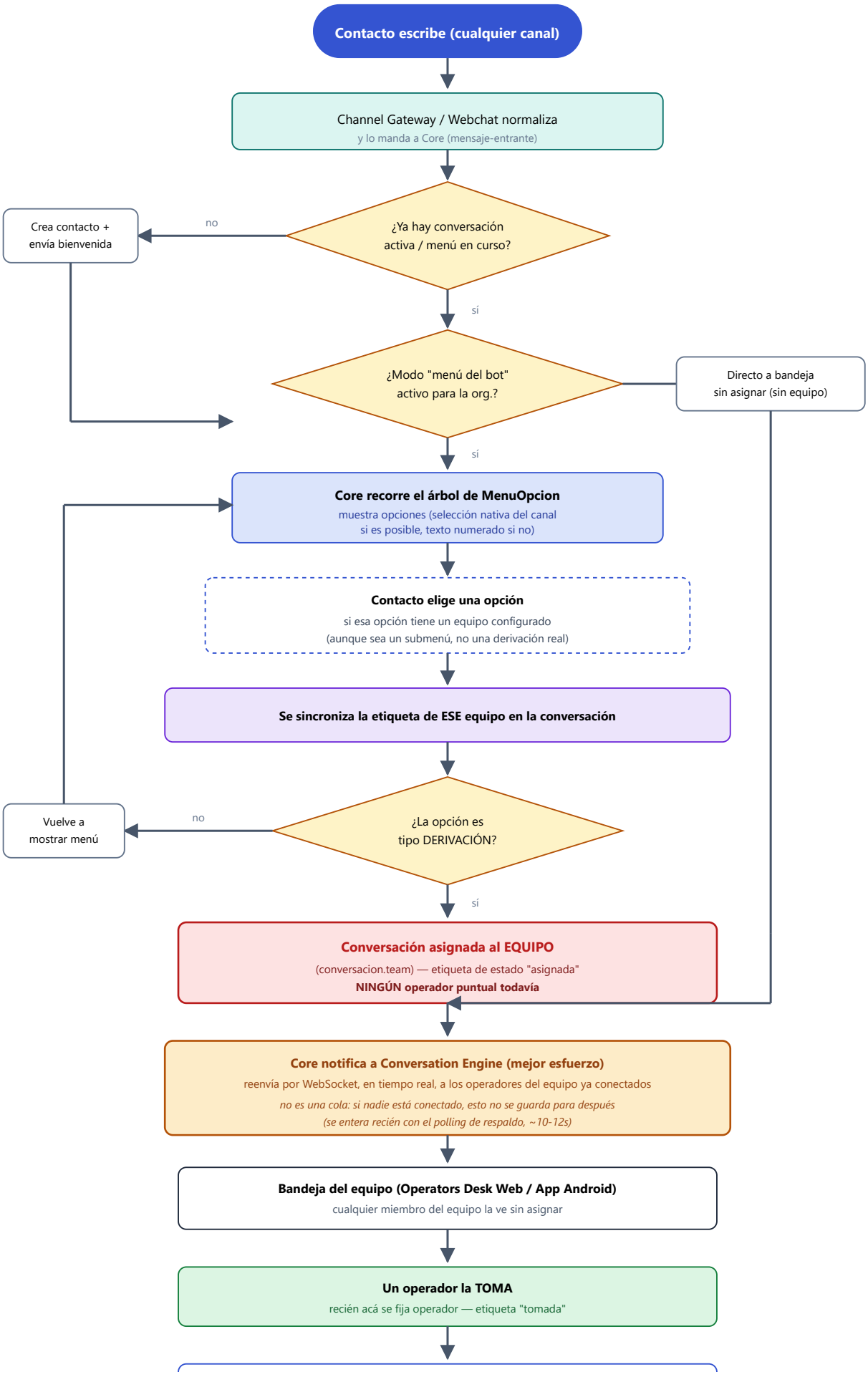
ANALYTICS NO ESTÁ EN ESTA TABLA A PROPÓSITO

Se sacó del catálogo de "lo que existe hoy" — es sólo un `/health` sin una sola línea de lógica de reportes (8 líneas de código), así que listarlo junto a servicios reales exageraba cuánto hay construido. Pasa a la sección 9, junto con la integración con Telegram, como **funcionalidad para otra etapa**: algo planeado y con lugar reservado en la arquitectura (esquema de base, rol Data Manager, `ServiceToken`), pero no parte del alcance actual.

04

Flujo de una conversación, de punta a punta

Desde el primer mensaje de un contacto hasta que un operador humano la cierra. La regla de negocio más importante de todo el flujo, y la más corregida durante el desarrollo: **la derivación asigna la conversación a un EQUIPO, nunca automáticamente a un operador puntual** — queda en la bandeja del equipo hasta que alguien la toma a mano.


Contacto escribe (cualquier canal)
Channel Gateway / Webchat normaliza
y lo manda a Core (mensaje-entrante)
¿Ya hay conversación activa / menú en curso?
no
Crea contacto + envía bienvenida
sí
¿Modo "menú del bot" activo para la org.?
Directo a bandeja sin asignar (sin equipo)
sí
Core recorre el árbol de MenuOpcion
muestra opciones (selección nativa del canal si es posible, texto numerado si no)
Contacto elige una opción
si esa opción tiene un equipo configurado (aunque sea un submenú, no una derivación real)
Se sincroniza la etiqueta de ESE equipo en la conversación
¿La opción es tipo DERIVACIÓN?
no
Vuelve a mostrar menú
sí
Conversación asignada al EQUIPO
(conversacion.team) — etiqueta de estado "asignada"
NINGÚN operador puntual todavía
Core notifica a Conversation Engine (mejor esfuerzo)
reenvía por WebSocket, en tiempo real, a los operadores del equipo ya conectados
no es una cola: si nadie está conectado, esto no se guarda para después
(se entera recién con el polling de respaldo, ~10-12s)
Bandeja del equipo (Operators Desk Web / App Android)
cualquier miembro del equipo la ve sin asignar
Un operador la TOMA
recién acá se fija operador — etiqueta "tomada"

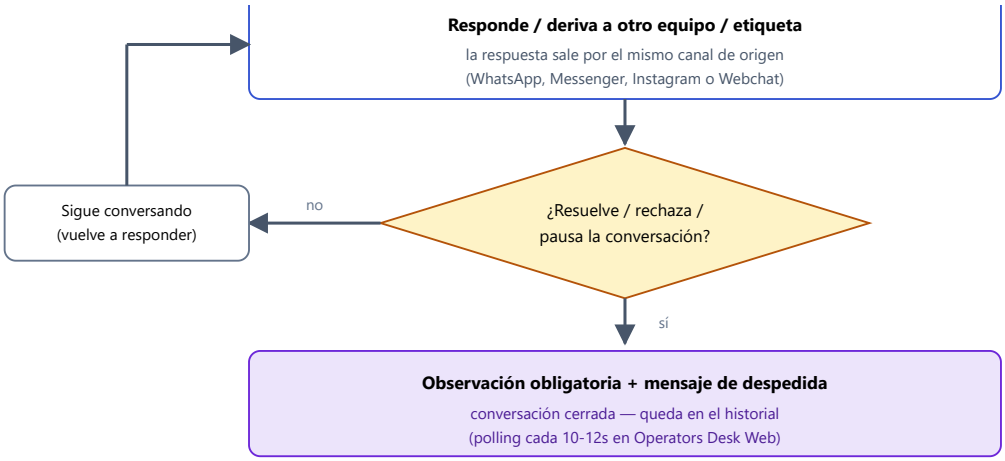


Figura 2 — Flujo de una conversación. Los rombos son decisiones; el recuadro rojo marca el punto exacto donde la conversación queda "del equipo" sin ningún operador todavía; el recuadro ámbar muestra dónde entra Conversation Engine — un relay de notificaciones en tiempo real, no una cola con persistencia.

Las dos reglas que más costó afinar

- **La etiqueta de equipo no espera a la derivación real.** `MenuOpcion.team` es un campo libre en cualquier tipo de nodo, no sólo en Derivación — si un Admin lo carga en un submenú ("esto es el área de Soporte"), la conversación recibe esa etiqueta desde que el contacto entra ahí, aunque el árbol nunca llegue a derivar de verdad.
- **Derivar nunca asigna un operador puntual.** El round-robin automático que existía al principio se quitó a pedido explícito de negocio — `conversations.asignacion. asignar_round_robin` sigue probado y disponible, pero ningún punto de la aplicación lo llama hoy. Asignar a una persona es siempre una acción humana ("tomar").

05

Autenticación, autorización y multi-tenancy

Dos esquemas de autenticación, nunca mezclados

Esquema	Header	Para quién	Detalle
JWT (RS256)	`Authorization: Bearer <jwt>`	Personas (Admin Panel, App Android)	Emitido por Core, incluye `org_id` y `groups` (lista, no un rol único). Validado contra `/api/auth/jwks/`.
ServiceToken	`Authorization: Service-Token <token>`	Microservicios entre sí	Un token por servicio (Channel Gateway, Conversation Engine, Analytics, Webchat), cifrado en reposo (Fernet), comparado con `hmac.compare_digest`. Sin `User` de Django detrás — `request.user` queda anónimo.

Autorización: grupos, nunca un campo "rol"

`User` no tiene un campo `role` — los permisos vienen enteramente de grupos de Django (M2M, un usuario puede estar en más de uno). Cinco grupos: `superusuarios`, `administradores`, `encargados_difusion`, `analistas_datos`, `operadores` — los últimos tres no tienen hoy ningún permiso sobre Core (sus pantallas viven en servicios que todavía no existen o se armaron directo en `conversations`). El frontend nunca decide qué mostrar mirando el nombre de un grupo: siempre consulta `permissions` (la lista real de `user.get_all_permissions()`) desde `GET /api/core/users/me/`.

Multi-tenancy: aislamiento por organización, sin excepción

Todo ViewSet org-scoped hereda de `OrganizationScopedModelViewSet`, que filtra por la organización del usuario autenticado automáticamente. Un superusuario real de plataforma (`is_superuser=True` + sin organización) **no** tiene un bypass para ver todo sin filtrar — ese atajo existió y se quitó deliberadamente: hoy gestiona sólo lo que es genuinamente de toda la plataforma (organizaciones, usuarios de cualquier org., tokens de servicio, estadísticas, salud de canales), nunca la configuración puntual de una organización (equipos, menú, canales, etiquetas).

Secretos: nunca en texto plano, ni en la respuesta ni en la base

- `Canal.api_key`, `ServiceToken.token` y `webhook_secret` se guardan cifrados en la base (Fernet, `FIELD_ENCRYPTION_KEY`).
- La API nunca devuelve el secreto completo salvo una vez, al generarlo/rotarlo — el resto del tiempo sólo `*_configurada` (bool) + `*_preview` (últimos 4 caracteres).
- Login con bloqueo por IP (3 fallas → 12 h), recuperación de contraseña y de usuario con mensaje anti-enumeración (siempre la misma respuesta, exista o no la cuenta).

06

Modelo de datos por servicio

Resumen de entidades — el detalle completo de cada campo vive en `docs/CONTEXTO_PROYECTO.md` §4; acá sólo el mapa para ubicarse.

core

- `Organization`, `User` (+grupos), `Team`
- `MenuOpcion` (árbol recursivo), `HorarioAtencion`, `DiaExcepcional`
- `Conversacion`, `ConversacionLog`, `ConversacionEtiqueta`, `ConversacionObservacion`, `ConversacionNotaInterna`
- `Contact`, `Canal`, `Etiqueta`, `BotMessages`, `ServiceToken`

channel_gateway

Sin modelos propios de configuración — pide `api_key` / `webhook_secret` a Core en cada request.

conversation_engine

Sin tablas de negocio propias — sólo registro en memoria de conexiones WebSocket activas (no persistido).

webchat

Sin Postgres propio todavía — sesiones de visitante y conexiones WebSocket en memoria del proceso.

analytics (futura etapa)

Todavía no tiene esquema propio — leería de `core` / `conversation_engine` con un usuario read-only cuando se construya (sección 9).

07

Convenciones multicanal

Dos reglas transversales que aplican a cualquier canal nuevo que se agregue a futuro:

Selección nativa, no texto numerado, cuando el canal lo permite

Core nunca decide el formato de UI de un canal — devuelve una lista estructurada de opciones (`opciones: [{id, titulo}]`) y cada Channel Gateway-type service la traduce a su widget nativo: WhatsApp usa `interactive.type=list` (4-10 opciones) o `button` (1-3); fuera de esos límites, o en canales sin ese soporte, cae al texto plano numerado que Core ya arma como fallback universal.

Marcado neutro, nunca la sintaxis literal de un canal

Todo texto configurado en Core usa marcado neutro (`**negrita**` , `_cursiva_` , `~tachado~` , `` `mono` ``) — jamás el asterisco simple de WhatsApp o el MarkdownV2 de Telegram guardado directamente. Cada adaptador de canal traduce (o descarta, si el canal no soporta texto enriquecido) al armar el mensaje saliente.

08

Estado de implementación por fase

Fase	Qué cubre	Estado
0 — Esqueleto	Compose, healthchecks, esquemas de Postgres	Completa
1 — Organizaciones/usuarios/equipos	Login, grupos, ABM, seguridad de login	Completa
2 — Menú y horarios (portado)	Árbol de menú, horarios, etiquetas, canales, modo con/sin menú	Completa
3 — Channel Gateway + WhatsApp	Webhooks Meta, envío saliente, selección nativa	Completa
4 — Conversation Engine	Entrega en tiempo real por WebSocket	Parcial — sin reconexión, un solo proceso
5 — Atención (Operators Desk)	Bandeja/tomar/derivar/etiquetar/cerrar	Completa — Operators Desk Web + Core
6 — App Android	Mismo contrato que Operators Desk Web	Completa — sin refresh token ni push
8 — QA end-to-end y hardening	Aislamiento multi-tenant, carga, revisión de lógica	No iniciada
Webchat (fast-follow)	Widget embebible como canal adicional	Completa — primera versión, integrada

> El detalle de qué falta para que esto sea un sistema en producción (no sólo "funcionalmente completo") está en el documento separado **"Qué falta para producción"** — seguridad de configuración, observabilidad, despliegue, cobertura de tests y continuidad operativa.

Funcionalidades para otra etapa (fuera del alcance actual)

No es deuda ni algo a medio hacer — es trabajo que conscientemente no se empezó todavía, al mismo nivel que agregar un canal nuevo. Se agrupan acá para no mezclarlas con el estado de lo que sí se construyó arriba:

Funcionalidad	Qué agregaría	Por qué no está ahora
Analytics (Fase 7)	Reportes de volumen/tiempos de respuesta/uso de menú para el rol Data Manager, export CSV.	Sin demanda todavía del rol que lo usaría — el servicio ya tiene puerto, esquema de `ServiceToken` y rol reservados, sólo falta construirlo.
Integración con Telegram	Un canal más además de WhatsApp/Messenger/Instagram/Webchat — mismo contrato normalizado contra Core, inline keyboard como selección nativa (sección 8).	Ningún cliente lo pidió todavía; la arquitectura ya está pensada para sumar un canal sin tocar Core (sólo un adaptador nuevo, como Channel Gateway o Webchat Service).
Push notifications (Fase 9)	Notificar a la app Android en segundo plano (FCM) cuando le llega una conversación nueva.	Fuera de alcance sin pedido explícito — hoy la app depende de tenerla abierta (polling + WebSocket).
Editor de menú avanzado (Fase 9)	Nodos de captura de datos del contacto y autorespuesta por palabra clave libre en el árbol de menú.	Fuera de alcance sin pedido explícito — el árbol hoy sólo tiene los tipos de nodo portados del repo original.

09

Glosario

Organización	Un cliente/tenant de la plataforma. Todo dato de negocio cuelga de una.
Equipo (Team)	Grupo de operadores dentro de una organización; unidad real de asignación de conversaciones.
Canal (Canal)	Una integración de mensajería configurada (WhatsApp/Messenger/Instagram/Webchat) para una organización.
MenuOpcion	Un nodo del árbol de menú del bot (submenú, respuesta, derivación, volver, inicio, terminar).
Conversación	El hilo entre un contacto y la organización — pasa por pendiente → asignada → tomada → resuelta/rechazada/en pausa.
Etiqueta de sistema	Estado de la conversación (pendiente/asignada/tomada/...), mutuamente excluyente entre sí.
Etiqueta de equipo	Qué equipo "tocó" la conversación — se sincroniza sola, desacoplada del campo real de asignación.
ServiceToken	Credencial de un microservicio para autenticarse contra Core (no es una api key de canal externo).
Derivación	El contacto llegó, dentro del árbol de menú, a un nodo que asigna la conversación a un equipo.